Agentic Microservices

EKS Deployment & Application Architecture

Amazon EKS · Kubernetes · ALB Ingress · Spring Boot · FastAPI · LangGraph · Observability

Platform	Namespaces	Services
Amazon EKS (Fargate-ready)	ecommerce · observability	9 (3 Java + 1 Python + 4 Obs + 1 Daemon)

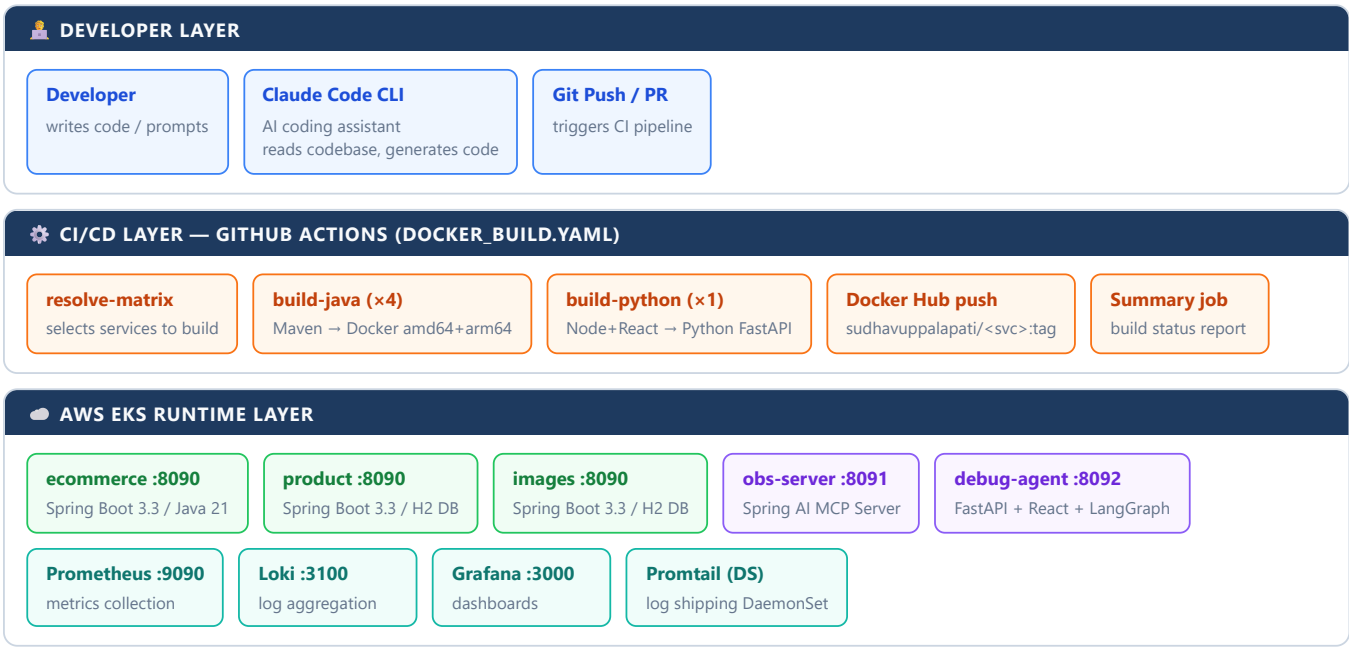
CI/CD	AI Layer	Prepared by
GitHub Actions + Docker Hub	Claude Code + LangGraph + OpenAI	vuppalapatisn

Table of Contents

1 · Architecture Overview & CI/CD Pipeline

System Summary: Agentic Microservices is a cloud-native e-commerce platform with an AI-powered observability layer. Five custom microservices (3 Java Spring Boot + 1 Python FastAPI + 1 Spring AI MCP Server) are deployed on Amazon EKS alongside a full observability stack (Prometheus, Loki, Grafana). An AI debug agent powered by LangGraph and OpenAI gpt-4.1-mini provides natural-language root-cause analysis backed by live telemetry.

1.1 High-Level Component Map



1.2 Claude Code Integration Flow

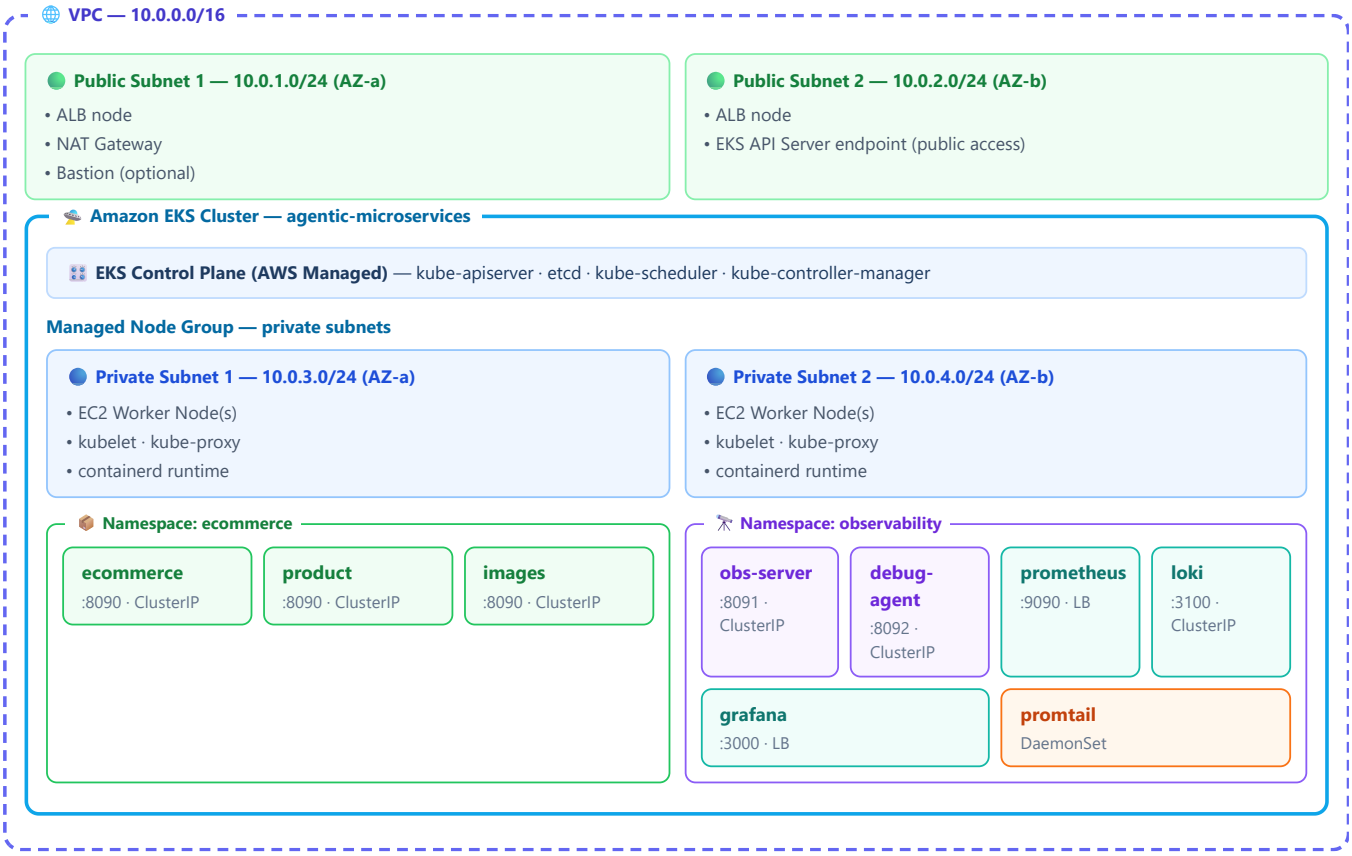
How Claude Code drives development: The developer streams requirements as structured JSON via `claude --input-format stream-json`. Claude Code reads the full repository — `microservices/`, `k8s/`, `CF/`, `.github/workflows/` — then generates or modifies Java/Python source, Dockerfiles, Kubernetes manifests, and CloudFormation templates, always following existing conventions (multi-arch Docker, non-root users, Spring Boot actuator probes). The resulting commit triggers `docker_build.yaml` which builds only the changed services and pushes new images to Docker Hub.

Step	Actor	Action	Output
1	Developer	Describes feature / fix in natural language	Prompt / JSON input
2	Claude Code	Reads repo, generates/edits code files	Modified source files
3	Developer	Reviews diff, commits, pushes to master	Git push event
4	GitHub Actions	resolve-matrix → build-java / build-python	Multi-arch Docker images
5	Docker Hub	Stores `sudhavuppalapati/<svc>:sha-xxx`	Image registry
6	CloudFormation / kubectl	Deploys updated manifests to EKS	Rolling update on EKS
7	Debug Agent	Developer asks "why is latency high?" in chat UI	AI root-cause analysis

2 · EKS Deployment Architecture

Infrastructure: The platform runs on Amazon EKS inside a dedicated VPC with public and private subnets across two availability zones. An AWS ALB Ingress Controller (running inside the cluster) manages a single internet-facing ALB shared across all services. Worker nodes run in private subnets and reach the internet via a NAT Gateway for Docker Hub image pulls and OpenAI API calls.

2.1 VPC & Networking Layout



2.2 Required EKS Add-ons

Add-on	Purpose	Notes
AWS ALB Ingress Controller	Creates and manages the internet-facing ALB from Ingress objects	Requires IRSA with `AWSLoadBalancerControllerIAMPolicy`
AWS VPC CNI	Assigns real VPC IPs to pods (enables ALB target-type: ip)	Default EKS managed add-on
CoreDNS	Resolves `<svc>.<ns>.svc.cluster.local` for inter-service calls	Default EKS managed add-on
kube-proxy	Maintains iptables rules for ClusterIP services	Default EKS managed add-on
EBS CSI Driver	Provides PersistentVolumes for Prometheus / Loki (production)	Recommended — current setup uses ephemeral storage

⚠️ **Production note:** Prometheus and Loki currently use ephemeral pod storage. Add EBS PersistentVolumeClaims (gp3) backed by the EBS CSI Driver to prevent data loss on pod restarts.

3 · Application Architecture & Service Communication

3.1 Request Flow: External → ALB → Services

3.2 Inter-Service Communication (Kubernetes CoreDNS)

Caller	Called Service	DNS Name	Config Key
ecommerce pod	product pod	`product-service.ecommerce.svc.cluster.local:8090`	SERVICES_PRODUCT_BASE_URL
ecommerce pod	images pod	`images-service.ecommerce.svc.cluster.local:8090`	SERVICES_IMAGES_BASE_URL
debug-agent pod	obs-server pod	`observability-server.observability.svc.cluster.local:8091`	OBSERVABILITY_AGENT_BASE_URL
obs-server pod	prometheus pod	`prometheus.observability.svc.cluster.local:9090`	OBSERVABILITY_PROMETHEUS_BASE_URL
obs-server pod	loki pod	`loki.observability.svc.cluster.local:3100`	OBSERVABILITY_LOKI_BASE_URL
debug-agent pod	grafana pod	`grafana.observability.svc.cluster.local:3000`	GRAFANA_API_BASE_URL

3.3 AI Debug Agent Flow (MCP Protocol)

3.4 Service Descriptions

Service	Stack	Port	Context Path	Responsibility
ecommerce	Java 21	`8090`	`/ecommerce-service`	Core e-commerce logic — orchestrates calls to product and images services; REST API for product orders and coupon integration
product	Java 21	`8090`	`/product-service`	Product catalog — provides product details and pricing; H2 in-memory database
images	Java 21	`8090`	`/image-service`	Image management — stores and serves product images; H2 in-memory database
observability-server	Spring AI	`8091`	`/actuator`	MCP Server — exposes Prometheus metrics and Loki logs as AI-callable tools via OpenAI function-call interface
observability-debug-agent	Python 3.12	`8092`	`/observability`	AI Debug Agent — React/Vite chat UI + FastAPI backend + LangGraph. Calls MCP Server to answer developer questions about production behaviour
prometheus	Observability	`9090`	`/prometheus`	Metrics collection — scrapes JVM and HTTP metrics from all three business services every 15 seconds
loki	Observability	`3100`	`/loki`	Log aggregation — receives structured JSON logs from Promtail; queried by Grafana and the MCP Server
grafana	Observability	`3000`	`/grafana`	Dashboards — pre-configured with Prometheus and Loki datasources; ecommerce-observability dashboard built in
promtail	DaemonSet	`9080`	—	Log shipping — mounts /var/log/pods on every node, parses CRI format logs, pushes to Loki

4 · ALB Ingress Routing

Ingress Strategy: Two Ingress objects (one per namespace) share a single AWS ALB via the annotation `alb.ingress.kubernetes.io/group.name: agentic-microservices`. The ALB uses **target-type: ip** (routes directly to pod IPs, no NodePort overhead) and **scheme: internet-facing**.

4.1 Ingress Object: ecommerce namespace

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ecommerce-ingress
  namespace: ecommerce
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP":80}]'
    alb.ingress.kubernetes.io/group.name: agentic-microservices
```

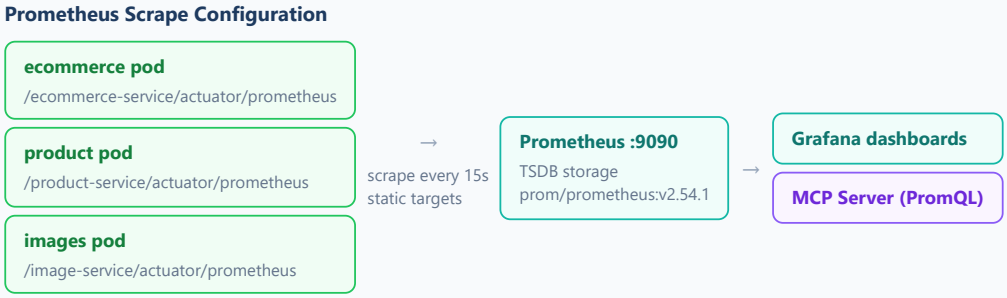
4.2 ALB Routing Table

Path Prefix	Namespace	Backend Service	Port	Group	Health Check
`/ecommerceApp*`	ecommerce	`ecommerce-service`	8090	agentic-microservices	`/actuator/health`
`/product-service*`	ecommerce	`product-service`	8090	agentic-microservices	`/actuator/health`
`/image-service*`	ecommerce	`images-service`	8090	agentic-microservices	`/actuator/health`
`/observability*`	observability	`observability-debug-agent`	8092	agentic-microservices	`/health`

⚠ **HTTPS:** Add `alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:...` and `alb.ingress.kubernetes.io/ssl-redirect: '443'` to enable TLS termination via ACM.

5 · Observability Architecture

5.1 Metrics Pipeline

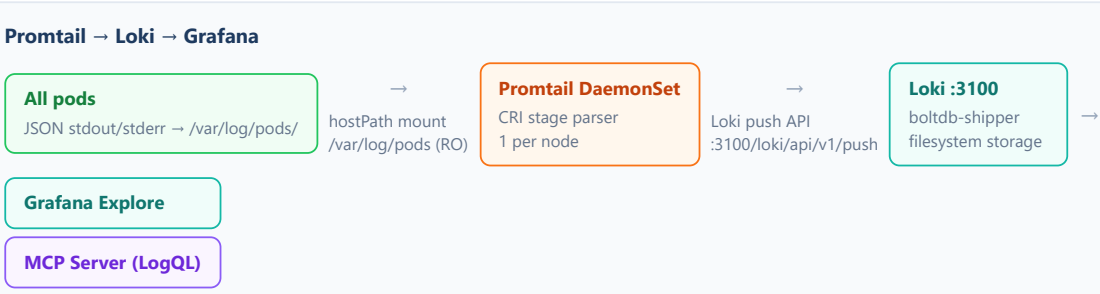


Metrics Retained After Relabeling

Metric	Description	Panel
`jvm_memory_used_bytes{area="heap"}`	Current heap usage in bytes	Heap Space
`jvm_memory_max_bytes{area="heap"}`	Maximum heap capacity	Heap Space
`jvm_threads_live_threads`	Number of live JVM threads	Thread Count
`jvm_gc_pause_seconds_*`	GC pause duration histogram	GC Activity
`http_server_requests_seconds_count`	HTTP request rate counter	Request Rate

Note: non-heap JVM memory metrics are dropped via Prometheus relabeling rules.

5.2 Logging Pipeline



Log Streams Collected by Promtail

Log Stream	Namespace	Log Path Pattern
ecommerce-service	ecommerce	`/var/log/pods/ecommerce_ecommerce-*/ecommerce/*.log`
product-service	ecommerce	`/var/log/pods/ecommerce_product-*/product/*.log`
images-service	ecommerce	`/var/log/pods/ecommerce_images-*/images/*.log`
observability-server	observability	`/var/log/pods/observability_observability-server-*/.../*.log`
observability-debug-agent	observability	`/var/log/pods/observability_observability-debug-agent-*/.../*.log`
grafana	observability	`/var/log/pods/observability_grafana-*/.../*.log`
loki	observability	`/var/log/pods/observability_loki-*/.../*.log`
prometheus	observability	`/var/log/pods/observability_prometheus-*/.../*.log`

5.3 Pre-built Grafana Dashboard — ecommerce-observability

Panel	Type	Query
Heap Space	Time series	`sum(jvm_memory_used_bytes{job="ecommerce",area="heap"})` vs `sum(jvm_memory_max_bytes{...})`
Request Rate	Time series	`sum(rate(http_server_requests_seconds_count{job="ecommerce"}[1m]))`
Thread Count	Stat	`jvm_threads_live_threads{job="ecommerce"}`
GC Activity	Time series	`rate(jvm_gc_pause_seconds_sum{job="ecommerce"}[5m])`

Dashboard UID: `ecommerce-observability` · Refresh: 10s · Time range: last 15 minutes · Anonymous access enabled

6 · Kubernetes Resource Reference

6.1 Full Resource Inventory

Pod	Namespace	Kind	Replicas	Image	CPU req/lim	Mem req/lim	Port	Service Type
🛒 ecommerce	ecommerce	Deployment	1	`sudhavuppalapati/ecommerce:latest`	100m / 500m	256Mi / 512Mi	8090	ClusterIP
📦 product	ecommerce	Deployment	1	`sudhavuppalapati/product:latest`	100m / 500m	256Mi / 512Mi	8090	ClusterIP
🖼️ images	ecommerce	Deployment	1	`sudhavuppalapati/images:latest`	100m / 500m	256Mi / 512Mi	8090	ClusterIP
🖥️ obs-server	observability	Deployment	1	`sudhavuppalapati/observability-server:latest`	100m / 300m	256Mi / 512Mi	8091	ClusterIP
💬 debug-agent	observability	Deployment	1	`sudhavuppalapati/observability-debug-agent:latest`	100m / 300m	256Mi / 512Mi	8092	ClusterIP
📊 prometheus	observability	Deployment	1	`prom/prometheus:v2.54.1`	100m / 300m	256Mi / 512Mi	9090	LoadBalancer
📜 loki	observability	Deployment	1	`grafana/loki:3.1.1`	100m / 300m	256Mi / 512Mi	3100	ClusterIP
📈 grafana	observability	Deployment	1	`grafana/grafana:11.1.4`	100m / 300m	256Mi / 512Mi	3000	LoadBalancer
📡 promtail	observability	DaemonSet	1/node	`grafana/promtail:3.1.1`	50m / 200m	128Mi / 256Mi	9080	None

6.2 Health Probe Configuration

Service	Startup Probe	Readiness Probe	Liveness Probe
ecommerce	—	/ecommerce-service/actuator/health (delay:20s, period:10s)	/ecommerce-service/actuator/health (delay:180s, period:15s)
product	/product-service/actuator/health (period:5s, fail:36)	/product-service/actuator/health (period:5s, fail:3)	/product-service/actuator/health (delay:180s, period:15s, fail:4)
images	/image-service/actuator/health (period:5s, fail:36)	/image-service/actuator/health (period:5s, fail:3)	/image-service/actuator/health (delay:180s, period:15s, fail:4)
obs-server	—	/actuator/health (delay:5s, period:10s)	/actuator/health (delay:20s, period:15s)
debug-agent	/health (period:5s, fail:24)	/health (period:10s)	/health (delay:20s, period:15s)

6.3 RBAC Configuration

ServiceAccount	ClusterRole	Permissions Granted
`prometheus` (observability)	prometheus	nodes, nodes/proxy, services, endpoints, pods — get/list/watch; configmaps — get; ingresses — get/list/watch
`promtail` (observability)	promtail	nodes, nodes/proxy, namespaces, pods, services, endpoints — get/list/watch

7 · ConfigMaps & Secrets Reference

7.1 ConfigMaps

ecommerce-config (namespace: ecommerce)

SERVER_PORT	8090
CONTEXT_PATH	/ecommerce-service
SERVICES_PRODUCT_BASE_URL	http://product-service:8090
SERVICES_IMAGES_BASE_URL	http://images-service:8090
SERVICES_COUPON_BASE_URL	http://coupon-service:8090
USE_IMAGES	true
MGMT_ENDPOINTS	health,info,prometheus

product-config / images-config (namespace: ecommerce)

SERVER_PORT	8090
DATASOURCE_URL	jdbc:h2:mem:productsdb
JPA_DDL_AUTO	none
SQL_INIT_MODE	always
JAVA_TOOL_OPTIONS	-XX:TieredStopAtLevel=1 -Xms128m -Xmx400m
MGMT_ENDPOINTS	health,info,prometheus

observability-server-config (namespace: observability)

LOKI_BASE_URL	http://loki.observability.svc.cluster.local:3100
LOKI_TIMEOUT_SECONDS	5
PROMETHEUS_BASE_URL	http://prometheus.observability.svc.cluster.local:9090
PROMETHEUS_TIMEOUT_SECONDS	5

observability-debug-agent-config (namespace: observability)

OPENAI_MODEL	gpt-4.1-mini
OBSERVABILITY_AGENT_BASE_URL	http://observability-server.observability.svc:8091
GRAFANA_API_BASE_URL	http://grafana.observability.svc:3000
GRAFANA_LOKI_DATASOURCE_UID	loki
GRAFANA_DASHBOARD_UID	ecommerce-observability
LANGGRAPH_DEBUG	true
REQUEST_TIMEOUT_SECONDS	10
STARTUP_VALIDATION_RETRIES	30

7.2 Secrets Required Before Deployment

Secret Name	Type	Namespace(s)	Key	Usage	Required?
`dockerhub-registry-secret`	kubernetes.io/dockerconfigjson	ecommerce + observability	.dockerconfigjson	imagePullSecrets for all pod deployments pulling from Docker Hub	✅ Required
`observability-debug-agent-secret`	Opaque	observability	OPENAI_API_KEY	OpenAI API calls from the LangGraph debug agent (optional: true in manifest)	⚠️ Optional

8 · Deployment Commands & Runbook

8.1 Prerequisites

- AWS CLI configured with credentials for the target account and region
- kubectl configured to talk to the EKS cluster (`aws eks update-kubeconfig --name agentic-microservices`)
- AWS ALB Ingress Controller installed with IRSA (see AWS docs for AWSLoadBalancerControllerIAMPolicy)
- Docker Hub account credentials with access to `sudhavuppalapati/*` images
- OpenAI API key (for debug agent AI features)

8.2 Step-by-Step Deployment

```
# ── STEP 1: Create namespaces ──────────────────────────────
kubectl apply -f k8s/namespace.yaml
# Creates: ecommerce  and  observability  namespaces

# ── STEP 2: Docker Hub pull secret (repeat for both namespaces) ─
kubectl create secret docker-registry dockerhub-registry-secret \
  --docker-server=docker.io \
  --docker-username=<DOCKER_USERNAME> \
  --docker-password=<DOCKER_PASSWORD> \
  --namespace ecommerce

kubectl create secret docker-registry dockerhub-registry-secret \
  --docker-server=docker.io \
  --docker-username=<DOCKER_USERNAME> \
  --docker-password=<DOCKER_PASSWORD> \
  --namespace observability

# ── STEP 3: OpenAI API key secret ──────────────────────────
kubectl create secret generic observability-debug-agent-secret \
  --from-literal=OPENAI_API_KEY=sk-... \
  --namespace observability

# ── STEP 4: Deploy observability stack ─────────────────────
kubectl apply -f k8s/observability/namespace.yaml
kubectl apply -f k8s/observability/prometheus/
kubectl apply -f k8s/observability/loki/
kubectl apply -f k8s/observability/promtail/
kubectl apply -f k8s/observability/grafana/
# Wait for observability pods to be ready
kubectl wait --for=condition=Ready pod -l app=prometheus -n observability --timeout=120s
kubectl wait --for=condition=Ready pod -l app=loki -n observability --timeout=120s
kubectl wait --for=condition=Ready pod -l app=grafana -n observability --timeout=120s

# ── STEP 5: Deploy business microservices ──────────────────
kubectl apply -f k8s/product/
kubectl apply -f k8s/images/
kubectl apply -f k8s/ecommerce/
# product and images must be up before ecommerce calls them

# ── STEP 6: Deploy AI observability agents ─────────────────
kubectl apply -f k8s/observability-server/
kubectl apply -f k8s/observability-debug-agent/

# ── STEP 7: Apply ingress rules ────────────────────────────
kubectl apply -f k8s/ingress/ingress.yaml
# ALB Ingress Controller will create/configure the ALB (~60-90 seconds)

# ── STEP 8: Get ALB DNS name ───────────────────────────────
```

```
kubectl get ingress -n ecommerce
kubectl get ingress -n observability
```

8.3 Verify Deployment

```
# Check all pods are Running
kubectl get pods -n ecommerce
kubectl get pods -n observability

# Get ALB address and test endpoints
ALB=$(kubectl get ingress ecommerce-ingress -n ecommerce \
  -o jsonpath='{.status.loadBalancer.ingress[0].hostname}')

curl http://$ALB/ecommerceApp/actuator/health
curl http://$ALB/product-service/actuator/health
curl http://$ALB/image-service/actuator/health
curl http://$ALB/observability/health

# View Grafana dashboard (opens in browser)
echo "Grafana: http://$ALB/grafana"
echo "Debug Agent: http://$ALB/observability"
```

8.4 CI/CD Image Update Workflow

```
# After GitHub Actions pushes a new image, force EKS to pull it
kubectl rollout restart deployment/ecommerce -n ecommerce
kubectl rollout restart deployment/product -n ecommerce
kubectl rollout restart deployment/images -n ecommerce

# Watch rollout status
kubectl rollout status deployment/ecommerce -n ecommerce

# Or update image tag directly (pin to a specific SHA build)
kubectl set image deployment/ecommerce \
  ecommerce=sudhavuppalapati/ecommerce:sha-abc1234 \
  -n ecommerce
```